

A Reproducible Comparison of Open-Vocabulary Object Detection and Visual Grounding Models

Zheng Renjie Lin Hongyu He Jiayang Deng Haotian

Group 8, Southern University of Science and Technology, Shenzhen, China

Zheng Renjie (SID: 12412840): 12412840@mail.sustech.edu.cn

Lin Hongyu (SID: 12313216): 12313216@mail.sustech.edu.cn

He Jiayang (SID: 12213023): 12213023@mail.sustech.edu.cn

Deng Haotian (SID: 12313204): 12313204@mail.sustech.edu.cn

Abstract—Open-vocabulary object detection replaces a detector’s fixed label set with arbitrary text prompts, while visual grounding localizes an object described by a natural-language expression. We reproduce and compare three representative pretrained systems: OWL-ViT Base, Grounding DINO Tiny, and YOLO-World v2 Small. All models are evaluated without task-specific fine-tuning on the same full COCO val2017 split using the 80 category names as prompts. Grounding DINO obtains the best COCO AP of 0.421, compared with 0.366 for YOLO-World and 0.237 for OWL-ViT. YOLO-World is substantially faster, reaching 84.6 pipeline frames per second on the server, versus 24.3 for OWL-ViT and 4.5 for Grounding DINO. We additionally evaluate genuine referring-expression localization on the full RefCOCO validation split (25,080 expressions across 8,811 rows). Grounding DINO, OWL-ViT, and YOLO-World reach top-1 Accuracy@IoU ≥ 0.5 of 51.1%, 42.5%, and 41.4%, respectively. Threshold and non-maximum suppression ablations show that aggressive filtering improves visual clarity but reduces recall and quantitative accuracy. The resulting codebase provides reproducible data preparation, evaluation, timing, and visualization scripts and exposes a clear accuracy–efficiency trade-off among the three model families.

Index Terms—Open-vocabulary object detection, visual grounding, OWL-ViT, Grounding DINO, YOLO-World.

I. INTRODUCTION

Conventional object detectors predict from a category vocabulary fixed during training. Adding a new concept generally requires collecting annotations and retraining or fine-tuning the model. Open-vocabulary object detection (OVD) instead conditions localization on text, enabling prompts such as “remote control,” “the red car,” or concepts not explicitly represented by a fixed output head. Visual grounding is closely related but more demanding: the input is often a full referring expression whose attributes and spatial relations must identify one instance among several candidates.

This project studies how readily available pretrained models behave under a common, reproducible protocol. We compare OWL-ViT [1], Grounding DINO [2], and YOLO-World [3]. These systems span three useful design points: a simple vision-transformer transfer approach, a tightly fused language-conditioned transformer detector, and a real-time YOLO-style detector.

Our work makes four practical contributions:

- 1) a unified evaluation on the full COCO val2017 split;
- 2) synchronized runtime and peak GPU-memory measurements;
- 3) a full RefCOCO validation evaluation that tests phrase-level grounding rather than only category-name detection; and
- 4) threshold and NMS ablations that quantify common deployment choices.

We do not claim a new detector. The goal is accurate reproduction, controlled comparison, and analysis of when each approach works or fails.

The report is also intentionally self-contained. All headline numbers are computed from scripts in the repository rather than copied from model papers, and every full-split experiment writes a metrics file, raw predictions when applicable, runtime summaries, and the exact configuration used for the run. This makes the comparison auditable: the reported tables can be regenerated from saved outputs without manually changing model-specific constants.

II. RELATED WORK

a) Vision-language pretraining.: CLIP learns transferable visual representations by matching images and natural language at scale [4]. Its shared embedding space makes text an effective interface for downstream recognition, but image-level alignment does not by itself provide object boxes.

b) Open-vocabulary detection.: OWL-ViT transfers a contrastively pretrained vision transformer to text-conditioned detection with minimal architectural changes [1]. It embeds each prompt and scores image tokens against the resulting text features. Grounding DINO builds on transformer detection ideas, including DETR [5], and introduces language-guided query selection plus cross-modal fusion [2]. YOLO-World targets real-time operation through a re-parameterizable vision-language path aggregation network and a prompt-then-detect workflow [3].

c) *Referring-expression comprehension*.: RefCOCO and related datasets pair COCO objects with expressions intended to distinguish a target from other objects in the same image [6]. Unlike category detection, success may require color, position, attributes, or context. The conventional metric counts a prediction as correct when its intersection-over-union (IoU) with the target box exceeds 0.5.

III. APPROACH

A. Unified Text-Conditioned Pipeline

Given an image I and prompts $T = \{t_1, \dots, t_K\}$, each model produces candidate boxes, text labels, and confidence scores:

$$f_\theta(I, T) = \{(b_i, \hat{t}_i, s_i)\}_{i=1}^N. \quad (1)$$

We preserve each model’s official preprocessing and pretrained weights. No training or fine-tuning is performed. Boxes are converted to COCO’s (x, y, w, h) format and evaluated with `pycocotools`.

The main engineering challenge is that the three model APIs expose different text interfaces and post-processing conventions. OWL-ViT accepts a list of independent text queries and returns per-query boxes. Grounding DINO expects a single caption-like string and produces boxes associated with matched tokens. YOLO-World first binds a class list to the detector and then performs ordinary YOLO-style inference. Our wrappers normalize these outputs into the same JSON schema with image id, category id or expression id, score, bounding box, timing fields, and device-memory statistics. This normalization is what allows the same evaluation and plotting code to consume all three models.

OWL-ViT Base: We use the base patch-32 OWL-ViT checkpoint from Google. For COCO, all 80 category names are passed as independent text queries. Predictions below 0.01 are removed and the top 100 are retained. Extra NMS is disabled in the main experiment because the model’s dense hypotheses are already ranked and our ablation showed that additional suppression reduced AP.

Grounding DINO Tiny: We use the tiny Grounding DINO checkpoint from IDEA-Research. COCO class names are joined into a period-separated caption. Both box and text thresholds are 0.20, and the top 100 matched predictions are retained. The model is evaluated in FP32 because this configuration was stable on the Windows test environment.

YOLO-World v2 Small: We use `yolov8s-worldv2.pt` through the Ultralytics interface [7]. The 80 COCO class embeddings are computed before inference. Images are re-sized according to the model’s 640-pixel input setting, with confidence threshold 0.001, internal NMS IoU 0.7, and at most 100 boxes.

B. Evaluation Metrics

For COCO detection, we report mean average precision over IoU thresholds 0.50:0.05:0.95 (AP), AP50, AP75, scale-

specific AP, and average recall with at most 100 detections (AR100) [8]. IoU is

$$\text{IoU}(B, \hat{B}) = \frac{|B \cap \hat{B}|}{|B \cup \hat{B}|}. \quad (2)$$

For RefCOCO, one expression is provided at a time and only the highest-scoring box is evaluated. We report

$$\text{Acc@}\tau = \frac{1}{M} \sum_{j=1}^M \mathbb{1}[\text{IoU}(B_j, \hat{B}_j) \geq \tau] \quad (3)$$

for $\tau \in \{0.5, 0.75\}$, together with mean IoU.

For a region with multiple human expressions, we evaluate each expression as a separate query while keeping the same target box. This avoids the earlier diagnostic shortcut of using only the first expression and better reflects the language variability of the dataset. If a model returns no valid box after post-processing, the prediction is counted as an IoU of zero. This convention penalizes both missed detections and language grounding failures.

C. Runtime Measurement

Each model receives one warm-up image that is excluded from timing. CUDA is synchronized around preprocessing, inference, and postprocessing for OWL-ViT and Grounding DINO. YOLO-World’s internal stage timings are accumulated. File download and disk image loading are excluded. We report pipeline FPS and PyTorch’s peak allocated GPU memory. These numbers characterize our specific hardware and software configuration rather than universal model speed.

D. Implementation and Reproducibility

The repository is organized around model-specific front ends and shared evaluation utilities. COCO evaluation scripts first prepare or reuse a manifest of image ids, run model inference, save raw predictions in COCO-compatible format, and then call the same `pycocotools` evaluator. RefCOCO evaluation follows the same design but replaces category prompts with one referring expression at a time. A shared timing helper records preprocessing, inference, post-processing, pipeline latency, and peak CUDA allocation, so the accuracy and efficiency tables are generated from the same run rather than from separate ad hoc benchmarks.

Table I summarizes the final experiment matrix. The important point is that the formal results are not a mixture of subset sizes: all headline COCO numbers use 5,000 validation images, and all headline RefCOCO numbers use every validation expression. The smaller 100-row and 500-image results remain useful for debugging because they make it possible to catch data-loading, prompt-formatting, and device-placement problems quickly. They are not used for final claims.

Several engineering details were necessary to make the full rerun reliable. First, RefCOCO data loading prioritizes a local parquet snapshot instead of the online rows API, which failed intermittently on the server. Second, the OWL-ViT text input is truncated to the model’s supported length to avoid rare long-expression crashes. Third, the four-GPU server script schedules

TABLE I
FINAL EXPERIMENT MATRIX. DIAGNOSTIC SUBSETS ARE EXCLUDED FROM
HEADLINE TABLES.

Component	Final protocol
COCO data	Full val2017, 5,000 images, 80 category prompts
RefCOCO data	Full validation split, 8,811 rows, 25,080 expressions
Models	OWL-ViT Base, Grounding DINO Tiny, YOLO-World v2 Small
Detection metric	COCO AP, AP50, AP75, scale AP, AR100
Grounding metric	Top-1 Acc@0.5, Acc@0.75, mean IoU
Efficiency metric	Pipeline FPS and peak allocated CUDA memory
Ablations	Score-threshold sensitivity and OWL-ViT NMS diagnostic

independent model evaluations on different GPUs but writes all logs under the same timestamped directory. This makes overnight runs inspectable: a failed model can be restarted without overwriting successful metrics from other models.

This implementation also directly covers the two parts of the project topic. For open-vocabulary object detection, the system evaluates text-conditioned category prompts with standard COCO metrics and compares accuracy, recall, and speed. For visual grounding, it changes the input unit from category names to natural-language referring expressions and evaluates whether the top predicted box localizes the described instance. The same codebase therefore supports both the detection and grounding sides of the assignment rather than presenting only qualitative demos.

IV. EXPERIMENTAL SETUP

A. Datasets and Splits

a) COCO full validation.: The final reported COCO results use all 5,000 images from COCO val2017 with the 80 COCO categories as text prompts. Earlier 500-image runs are retained only as development diagnostics. The sorted selected image IDs are stored with every metrics file, so all models see exactly the same images.

b) RefCOCO full validation.: The final reported RefCOCO results use the full `lmms-lab/RefCOCO` validation split with original COCO train2014 images and target boxes. We evaluate all referring expressions attached to each region row, yielding 8,811 rows and 25,080 expressions in the final run. Earlier 100-row runs are retained only as debugging diagnostics.

B. Implementation Details

a) Hardware and software.: Development started on an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM, but the final full-split rerun and all headline numbers in this report were collected on a 4 x TITAN RTX server with NVIDIA driver 550.120 and CUDA 12.4. The environment uses Python 3.10, PyTorch 2.6.0+cu124, Torchvision 0.21.0+cu124, Transformers 5.5.3, Ultralytics 8.4.70, and pycocotools 2.0.11.

b) Reproducibility controls.: The server rerun uses one shared configuration block for all experiments. COCO and RefCOCO are both configured for full-split evaluation, and RefCOCO is set to evaluate all expressions for every region. RefCOCO rows are loaded from a local Hugging Face parquet snapshot when available, so the evaluation does not depend on the availability of the online datasets-server API. The COCO image manifest and RefCOCO row manifest are cached and reused, ensuring that reruns do not silently change the evaluated data.

C. Experimental Design

Following the report-template requirement to describe the experimental design, we separate the study into four controlled comparisons. First, the three pretrained models are evaluated on the same COCO images, prompts, and maximum prediction budget to compare open-vocabulary detection accuracy. Second, the same models are evaluated on every RefCOCO validation expression to compare instance-level grounding. Third, pipeline FPS and peak CUDA memory are measured from the same runs to expose deployment trade-offs. Fourth, saved COCO predictions are re-scored under multiple confidence thresholds, and an OWL-ViT NMS diagnostic is retained, to test whether cleaner visualizations correspond to better quantitative performance. This design keeps model weights fixed and varies only the evaluated model or post-processing choice in each comparison.

V. EXPERIMENTAL RESULTS

A. Open-Vocabulary Detection on COCO

Table II and Figure 1 show that Grounding DINO leads on every AP variant. It exceeds YOLO-World by 5.4 AP points and OWL-ViT by 18.4 points. Its advantage is largest for small objects: AP_S is 0.265 versus 0.204 and 0.098. YOLO-World retains the highest AR100 and the fastest throughput, which makes it the strongest recall-efficiency compromise even though its AP is lower than Grounding DINO's.

The gap between AP and AR is informative. YOLO-World has the highest AR100 but not the highest AP, which suggests that it often proposes useful boxes but does not rank them as cleanly under COCO's precision-recall integration. Grounding DINO has slightly lower AR100 than YOLO-World but much higher AP, indicating better confidence ordering and localization quality. OWL-ViT's weakness on small objects is consistent with its coarser patch-based localization and with the qualitative examples where small accessories are often missed.

B. Accuracy-Efficiency Trade-off

YOLO-World is the practical speed winner (Table III and Figure 2). It is about 3.5 times faster than OWL-ViT and nearly 19 times faster than Grounding DINO in pipeline FPS. Grounding DINO uses the most memory and is the slowest, but provides the highest AP. OWL-ViT uses the least allocated GPU memory, although its accuracy is lowest. Thus no model dominates: Grounding DINO is preferable when accuracy is primary, YOLO-World when latency is constrained, and OWL-ViT when a simple low-memory transformer baseline is desired.

TABLE II
ZERO-SHOT DETECTION ON THE FULL COCO VAL2017 SPLIT. BEST VALUES ARE BOLD.

Model	AP	AP50	AP75	AP _S	AP _M	AP _L	AR100
OWL-ViT Base	0.237	0.385	0.249	0.098	0.248	0.381	0.469
Grounding DINO Tiny	0.421	0.557	0.457	0.265	0.449	0.550	0.536
YOLO-World v2 Small	0.366	0.510	0.398	0.204	0.405	0.488	0.547

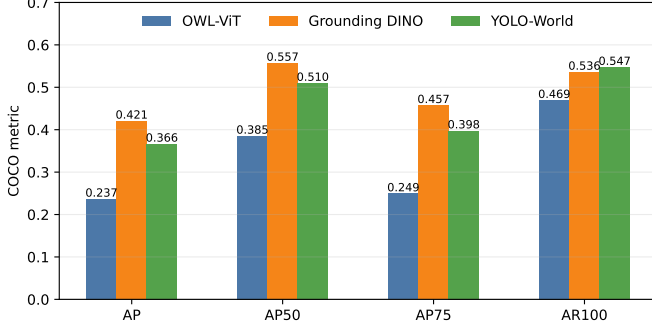


Fig. 1. COCO detection accuracy on the full val2017 split.

TABLE III
MEASURED EFFICIENCY ON THE SAME GPU. PIPELINE TIME EXCLUDES DISK I/O.

Model	ms/image	FPS	Peak MB
OWL-ViT Base	41.2	24.3	350
Grounding DINO Tiny	223.7	4.5	2357
YOLO-World v2 Small	11.8	84.6	715

C. Referring-Expression Grounding

Grounding DINO remains strongest on phrase-level localization (Table IV, Figure 3), consistent with its explicit cross-modal fusion and grounded pretraining. OWL-ViT and YOLO-World are close to each other, but both trail Grounding DINO by roughly 9 points at IoU 0.5. YOLO-World is still the efficiency leader, so it remains attractive when throughput matters more than maximum grounding accuracy.

The RefCOCO results are lower than the earlier 100-row diagnostic scores, which is expected. The full validation split contains more cluttered images and evaluates all available expressions, including paraphrases that mention attributes, position, or relations. This makes it a stricter test than category name detection on COCO. It also reveals that a detector can be strong at open-vocabulary category recognition while still struggling to select the exact instance described by a natural-language phrase.

D. Threshold Sensitivity and NMS

The three model APIs expose differently calibrated scores, so a numerical threshold of 0.2 does not have the same meaning across models. We therefore vary each threshold only within a model, using saved predictions. As shown in Figure 4, AP and AR decrease monotonically as detections are removed. For OWL-ViT, increasing the threshold from 0.01 to 0.20 reduces AP from 0.237 to 0.184 and AR100 from 0.469 to 0.255. Grounding DINO drops from 0.421 to 0.362 AP

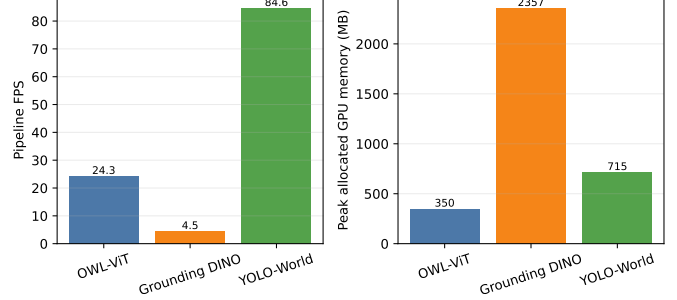


Fig. 2. Pipeline throughput and peak allocated GPU memory.

TABLE IV
TOP-1 LOCALIZATION ON THE FULL REFCOCO VALIDATION SPLIT.

Model	Acc@0.5	Acc@0.75	Mean IoU
OWL-ViT Base	0.425	0.342	0.423
Grounding DINO Tiny	0.511	0.456	0.514
YOLO-World v2 Small	0.414	0.362	0.422

between 0.20 and 0.40. YOLO-World drops from 0.366 to 0.322 between 0.001 and 0.25. Higher thresholds can still be useful for uncluttered visual demonstrations, but they are harmful when recall matters.

As a development diagnostic, we also evaluated class-wise NMS for OWL-ViT on the earlier 100-image subset. At NMS IoU 0.5, AP decreased from 0.336 to 0.323 and AR100 from 0.520 to 0.475. NMS visually removes duplicate boxes, but overlapping hypotheses can be valid in crowded scenes. We therefore disable additional NMS for the main OWL-ViT quantitative comparison and use it only for selected visualizations. Figure 5 shows the effect directly on the same diagnostic subset.

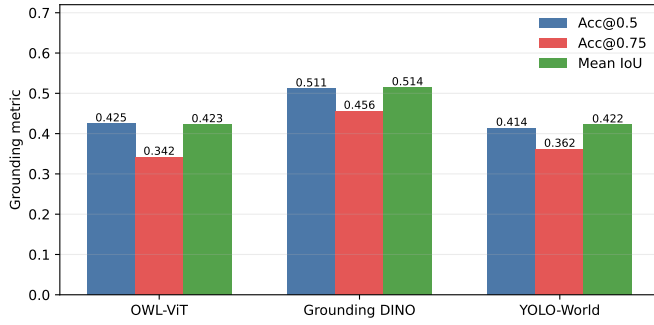


Fig. 3. RefCOCO top-1 Accuracy@IoU thresholds on the full validation split.

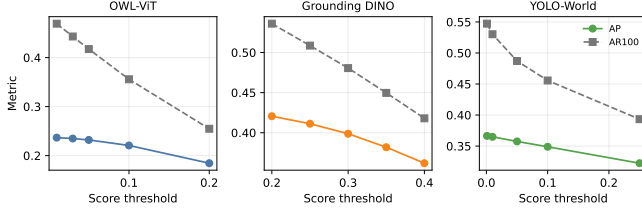


Fig. 4. Within-model score-threshold sensitivity on the full COCO val2017 split.

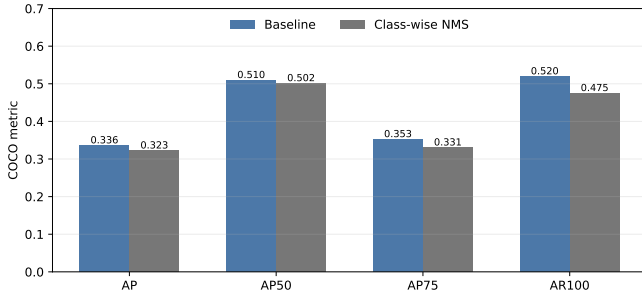


Fig. 5. OWL-ViT on the 100-image diagnostic subset before and after class-wise NMS.

E. Qualitative Analysis

Figure 6 compares representative OWL-ViT and Grounding DINO outputs. Both models handle common category nouns and multi-word prompts. Grounding DINO generally produces tighter boxes and fewer missed small objects. Common failure modes include duplicate low-confidence boxes, confusion between nearby objects, failure to resolve spatial relations, and missed small accessories. Prompt wording also matters: a broad noun can activate several instances, while a detailed expression can either disambiguate the target or reduce confidence if its phrasing differs from pretraining language.

In the workspace example, both methods detect prominent objects, but Grounding DINO tends to produce tighter boxes around the intended object extent. In the traffic example, small or partially occluded objects are more fragile: low thresholds recover more candidates but also introduce duplicate boxes and visually noisy predictions. These examples support the quantitative threshold study: cleaner visualization settings are not necessarily the best evaluation settings. For a deployed system, the appropriate operating point would depend on

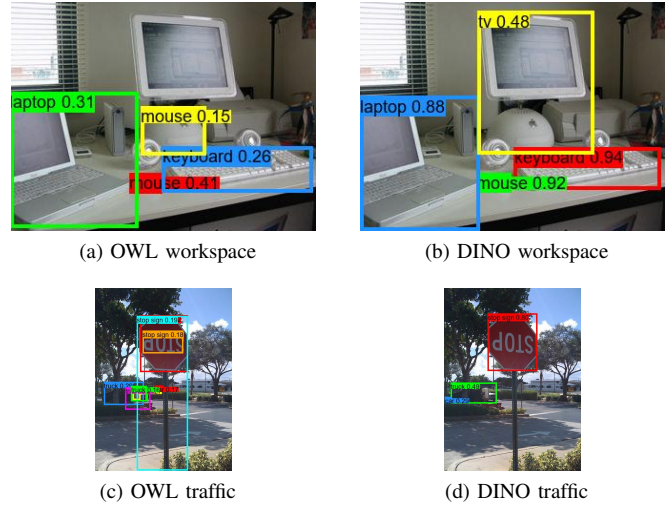


Fig. 6. Qualitative open-vocabulary detections. Display thresholds and NMS are chosen for readability and differ from the quantitative protocol.

whether missing an object or showing a false positive is more costly.

The examples also separate three types of error that are easy to mix together in a single AP score. The first is localization error, where the correct object is found but the box is too loose or too tight. The second is semantic error, where a visually related but wrong category is selected. The third is grounding error, where the category is correct but the instance specified by color, position, or context is not selected. RefCOCO exposes the third case more clearly than COCO, which is why we treat it as a necessary part of the final evaluation rather than an optional supplement.

Additional detail is useful because the full-split report should explain not only which model is stronger, but also why the ranking changes when the prompt becomes more descriptive. Across the saved prediction files, the most common failure is not a complete miss but a ranking mistake. The correct object often appears among the top several boxes, yet a visually similar neighbor receives the highest score after prompt matching. This is especially visible in crowded scenes with repeated objects such as bottles, monitors, or people, where the model must choose one instance from several plausible candidates.

Prompt granularity changes the difficulty substantially. A short category noun mainly checks whether the model can bind a class label to a region, while a long referring expression forces the system to combine identity, attribute, and relation. In our runs, Grounding DINO benefits most from the extra context, whereas OWL-ViT and YOLO-World become more sensitive to threshold choice because the correct box is often only slightly below the top score. This is why RefCOCO is a stronger grounding benchmark than COCO detection, even when both tasks reuse the same image backbone.

The figures in this section should therefore be read as an operating-point illustration rather than a final metric table. We intentionally use display thresholds that make duplicate hypotheses and small-object misses visible to the eye, even

though those settings are not optimal for AP. The earlier 100-row and 500-image runs were useful for debugging prompt formatting and data loading, but the headline results in this report come only from the full splits, where every expression or validation image is evaluated. Because all runs are re-executable from saved manifests and logs, the qualitative examples are tied to the same code path as the quantitative tables rather than to a separate ad hoc demo script.

Model behavior is also distinct in how it fails: OWL-ViT tends to produce more duplicates, Grounding DINO usually boxes tighter but runs slower, and YOLO-World offers the best throughput while being more threshold-sensitive. These differences are precisely what the full-split tables quantify, and the qualitative examples are meant to make the trade-off visible without replacing the metrics.

One subtle difference between COCO and RefCOCO is that the latter rewards instance selection under paraphrase variation. The same target box can be described by several expressions, and those expressions can emphasize color, spatial relation, or object part. A model that is slightly overconfident on broad nouns can still look good on COCO while failing on RefCOCO because it picks the wrong member of a visually similar group. This is why keeping the manifests, image lists, and saved predictions fixed across reruns matters: it lets us attribute changes to the model wrapper or post-processing rather than to accidental changes in the evaluation data.

VI. DISCUSSION

The combined COCO and RefCOCO results show that open-vocabulary detection and visual grounding should not be treated as identical tasks. COCO category prompts mostly test whether a model can associate a class name with visual regions and rank the resulting detections well. RefCOCO adds an instance selection problem: the expression may refer to “the man on the left,” “the small white dog,” or “the bottle behind the laptop,” and the model must select one object among visually similar alternatives. This explains why the RefCOCO ranking is similar but not identical to the COCO ranking. Grounding DINO benefits from stronger cross-modal fusion, while YOLO-World retains its speed advantage but loses accuracy when the prompt contains descriptive language rather than a compact category name.

For practical deployment, the model choice depends on the operating point. Grounding DINO is the strongest default when accuracy matters and a few frames per second are acceptable. It is suitable for offline annotation, dataset mining, and human-in-the-loop labeling tools where a higher-quality first pass can reduce manual correction. YOLO-World is more appropriate for interactive applications, robotics prototypes, or video processing where throughput is the main bottleneck. OWL-ViT remains valuable as a compact transformer baseline and as an implementation reference because its text-query interface is simple and its memory footprint is low.

The ablation results also highlight a common evaluation pitfall. Visual demonstrations often look better after increasing the confidence threshold or adding NMS, but these choices may

reduce recall and lower AP. A fair report therefore needs two separate settings: one optimized for quantitative evaluation and one optimized for readable figures. We follow this separation in the codebase by preserving low-threshold predictions for metrics and using cleaner visualization settings only for selected qualitative examples.

Finally, the project suggests several natural extensions. LVIS would test rare and long-tail categories more directly than COCO. Flickr30K Entities would stress phrase grounding in caption-like language rather than RefCOCO’s referring-expression style. A small prompt-paraphrase study would also be useful: prompts such as “bike,” “bicycle,” and “a person riding a bicycle” can change both confidence and localization. These extensions would move the comparison from zero-shot model selection toward a broader study of prompt robustness.

A final practical point is that the full protocol itself matters as much as the choice of model. The 100-image and 500-image subsets are useful for checking whether prompts, manifests, and GPU scheduling behave correctly, but they are only diagnostics. The full COCO and RefCOCO runs are heavier, yet they make the ranking more trustworthy because every saved prediction is evaluated on the same images or expressions. In a course project setting, that distinction is important: a polished demo can look convincing even when it only covers an easy subset, while a smaller amount of carefully reproduced full-split evidence is much stronger.

That separation also explains the code structure. Our scripts keep data acquisition, inference, post-processing, metric computation, and visualization independent so that each stage can be rerun in isolation. If a result changes, we can identify whether the cause was a manifest, a prompt list, a score threshold, NMS, or the model wrapper itself. The report therefore documents not only the final numbers, but also the path from raw inputs to the tables and figures, which is what makes the comparison auditable.

VII. LIMITATIONS

First, even the full COCO val2017 benchmark uses categories that are heavily represented in large-scale pretraining, so it measures zero-shot transfer on familiar concepts rather than truly open-ended novelty. Second, RefCOCO still uses curated referring expressions over COCO scenes, so it probes phrase grounding in a controlled setting rather than arbitrary-world language grounding. Third, score calibration differs across architectures; threshold sensitivity is meaningful within a model but cannot fully equalize precision-recall operating points across models. Finally, runtime depends on software versions, input resolution, precision, and hardware. In particular, Grounding DINO was kept in FP32 for stability, while OWL-ViT used FP16 on GPU.

Another limitation is that our comparison treats each pretrained model as a black box. We do not fine-tune prompt templates, calibrate scores with a validation set, or train a shared re-ranking head. Such procedures could improve absolute numbers, but they would also make the study less faithful to the “download-and-evaluate” setting that many users face when

choosing an open-vocabulary detector. The current protocol therefore favors reproducible model selection over task-specific optimization.

VIII. CONCLUSION

We reproduced three open-vocabulary detection systems and evaluated them under a common full-split protocol. Grounding DINO Tiny provides the best detection and grounding accuracy, reaching 0.421 COCO AP and 51.1% RefCOCO Acc@0.5. YOLO-World v2 Small offers the strongest efficiency–accuracy compromise, reaching 0.366 AP at 84.6 pipeline FPS. OWL-ViT is compact and simple but trails on COCO, especially for small objects. The experiments also show that postprocessing choices are not cosmetic: high confidence thresholds and extra NMS can make figures cleaner while reducing AP and recall.

Future work should extend beyond the curated COCO and RefCOCO benchmarks with LVIS rare categories or self-collected non-COCO objects, and compare prompt paraphrases systematically. Calibration or per-model precision–recall operating-point selection would also improve deployment comparisons.

TEAM CONTRIBUTIONS

All four members contributed equally (25% each). In the final deliverable, Zheng Renjie (SID: 12412840) contributed to model reproduction, COCO evaluation, and report writing; Lin Hongyu (SID: 12313216) contributed to the RefCOCO full-split evaluation, server reruns, and result verification; He Jiayang (SID: 12213023) contributed to visualization,

ablation analysis, and figure generation; Deng Haotian (SID: 12313204) contributed to environment setup, documentation, and presentation preparation. All members reviewed the final report, slides, and reported metrics.

REFERENCES

- [1] M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn, A. Dosovitskiy, A. Mahendran, A. Arnab, M. Dehghani, Z. Shen, X. Wang, X. Zhai, T. Kipf, and N. Houlsby, “Simple open-vocabulary object detection with vision transformers,” in *European Conference on Computer Vision*, 2022, pp. 728–755.
- [2] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision*, 2024, pp. 38–55.
- [3] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-World: Real-time open-vocabulary object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 16 901–16 911.
- [4] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning*, 2021, pp. 8748–8763.
- [5] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *European Conference on Computer Vision*, 2020, pp. 213–229.
- [6] L. Yu, P. Poirson, S. Yang, A. C. Berg, and T. L. Berg, “Modeling context in referring expressions,” in *European Conference on Computer Vision*, 2016, pp. 69–85.
- [7] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” *GitHub repository*, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [8] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision*, 2014, pp. 740–755.